Voice Control Object Tracking

1. Function Description

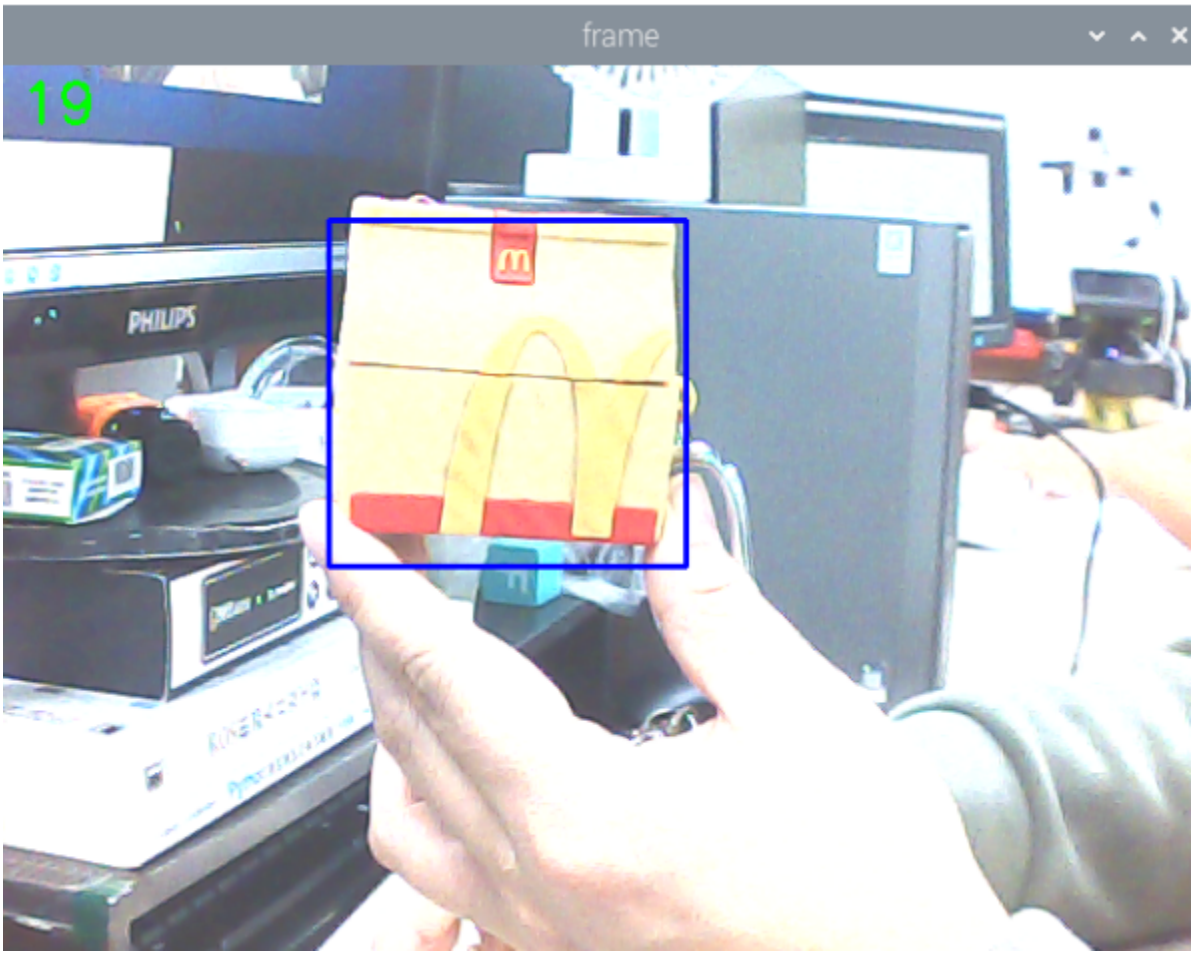
After the program runs, when a handheld object appears within the camera's view, use the mouse to frame the object to be tracked. Wake up the voice module through "Hello yahboom", then give the voice command "Tracking and picking up objects". The robotic arm will adjust its posture to keep the object's center near the screen center. When moving the object slowly, the robotic arm will follow the object's movement. Wake up the voice module through "Hello yahboom", give the voice command "stop following", and the robotic arm will cancel tracking the object and return to the initial posture.

2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start speech recognition and broadcast node
ros2 launch yahboom_speech speech.launch.py
# Start voice control object tracking node
ros2 run dofbot_voice_ctrl KCF_follow
```

After the program runs successfully, the robotic arm will move to the initial posture, and the handheld object appears within the camera's view. Use the mouse to frame the object to be tracked, as shown in the figure below:



Then, wake up the voice module through "Hello yahboom", then give the voice command "Tracking and picking up objects". When moving the object slowly, the robotic arm will follow the movement; after waking up with "Hello yahboom", the stop tracking voice command is: stop following, the robotic arm stops tracking and returns to the initial tracking posture.

3. Code Analysis

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/KCF/KCF_follow.py
```

Import necessary library files:

```
import cv2
import rclpy
from rclpy.node import Node
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
import threading
import time
import math
import os
from std_msgs.msg import Int8
from Arm_Lib import Arm_Device
import dofbot_follow.PID as PID
from  dofbot_follow.Track_Lib import *
```

Create robotic arm control object and control the robotic arm to move to initial posture:

```
self.Arm = Arm_Device()
self.init_joints = [90, 150, 0, 20, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Initialize PID:

```
self.xservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)
```

Create subscribers:

```
# Create subscriber to subscribe to image topic
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
# Create subscriber to subscribe to speech recognition result topic
self.sub_voice =
self.create_subscription(Int8,"voice_result",self.getVoiceResultCallBack,1)
```

Speech recognition result topic callback function getVoiceResultCallBack:

```
def getVoiceResultCallBack(self,msg):
    # If the received speech recognition result topic message is 107, change
self.start_flag value to True, indicating that tracking can be performed
    if msg.data == 107:
        self.start_follow = True
        print("Start tracking .")
    elif msg.data == 76:
    # If the received speech recognition result topic message is 76, execute
Reset() function, which includes changing self.start_follow value to False and
controlling the robotic arm to return to initial posture
        self.Reset()
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages, convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'bgr8')
    result_image = np.copy(rgb_image)
    #depth_image
    action = cv.waitKey(10) & 0xFF
    if action == 32:
        self.start_follow = True
    result_image = cv.resize(result_image, (640, 480))
    # Pass the image as parameter to process for framing the object and getting
the center point coordinates of the object's outer boundary. For this part, refer
to the code analysis in [3D Space Tracking]\[3.KCF Object Tracking].
    result_frame = self.process(result_image,action)
    # If the object center point coordinates are not 0 and self.start_follow
value is True, then tracking can be performed
    if self.cx!=0 and self.cy!=0 and self.start_follow == True:
        # If the object's center coordinates x or y are not within the center
range, execute XY_track function with the current object center coordinates as
parameters
        if (abs(self.cx-320) >10 or abs(self.cy-240)>10) :
            self.XY_track(self.cx,self.cy)
            print("Tracking")
            print("-------------------------------------")

    cv2.circle(result_frame, (int(self.cx),int(self.cy)), 10, (255,255,0),
thickness=None, lineType=None, shift=None)
    cur_time = time.time()
    fps = str(int(1/(cur_time - self.pr_time)))
    self.pr_time = cur_time
    cv2.putText(result_frame, fps, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,
255, 0), 2)
    cv.imshow(self.windows_name, result_frame)
```